Experiment 19: PCA and Explained Variance Analysis

Question

Problem Statement: A face-recognition attendance system stores thousands of pixels per face, making matching slow. The team needs fewer features without losing identity information. The Olivetti faces dataset (400 images, 64×64 pixels) or the handwritten digits dataset. The objective is to reduce dimensionality with PCA and choose components using explained variance. Use NumPy, Scikit-learn, Matplotlib.

Tasks:

1. Implement PCA from scratch using the covariance matrix and eigen-decomposition.
2. Verify the result against Scikit-learn's PCA.
3. Plot the scree plot and cumulative explained variance.
4. Find the number of components that retain 95% of variance.
5. Visualise the top eigenfaces and reconstructed faces with 10, 50 and 100 components.

Code

```
"""Experiment 19: PCA and Explained Variance Analysis (Olivetti faces)"""
import time
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.datasets import load_digits


np.random.seed(42)


# ---------------- Data: Olivetti faces (400 images, 64x64 = 4096 pixels) ----------------
try:
    from sklearn.datasets import fetch_olivetti_faces          # needs internet the first time
    X = fetch_olivetti_faces(shuffle=True, random_state=0).data
    IMG, NAME = (64, 64), "Olivetti faces"
except Exception as e:
    print("WARNING: could not load Olivetti faces (", type(e).__name__, ") -> using the digits dataset
instead.")
    X = load_digits().data
    IMG, NAME = (8, 8), "Digits"
n, d = X.shape
print(f"{NAME}: {n} images, {d} pixels each")
KS = [10, 50, 100] if d > 1000 else [5, 15, 30]               # numbers of components to compare


# ---------------- Task 1: PCA from scratch (covariance + eigen-decomposition) ----------------
t0 = time.time()
mean = X.mean(axis=0)
Xc = X - mean                                                  # 1. centre the data
cov = np.cov(Xc, rowvar=False)                                 # 2. covariance matrix (d x d)
eigvals, eigvecs = np.linalg.eigh(cov)                         # 3. eigen-decomposition (symmetric matrix)
order = np.argsort(eigvals)[::-1]                              # 4. sort by decreasing eigenvalue
eigvals, eigvecs = eigvals[order], eigvecs[:, order]
eigvals = np.clip(eigvals, 0, None)
print(f"From-scratch PCA took {time.time() - t0:.1f}s")


def project(Xc, k):    return Xc @ eigvecs[:, :k]              # 5. project onto top-k eigenvectors
def reconstruct(Xc, k):
    V = eigvecs[:, :k]
    return (Xc @ V) @ V.T + mean


# ---------------- Task 2: verify against scikit-learn ----------------
k_check = 20
skl = PCA(n_components=k_check, svd_solver="full").fit(X)
ratio_mine = eigvals / eigvals.sum()
print("\nVariance ratio (mine)    :", np.round(ratio_mine[:5], 5))
print("Variance ratio (sklearn) :", np.round(skl.explained_variance_ratio_[:5], 5))
print("Explained variance match :", np.allclose(ratio_mine[:k_check], skl.explained_variance_ratio_, atol=1e-
6))
same_axes = [np.isclose(abs(eigvecs[:, i] @ skl.components_[i]), 1.0, atol=1e-4) for i in range(k_check)]
print("Eigenvectors match (up to sign):", all(same_axes))
```

```
# ---------------- Task 3: scree plot and cumulative explained variance ----------------
cum = np.cumsum(ratio_mine)
fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
m = min(60, len(eigvals))
ax[0].plot(range(1, m + 1), eigvals[:m], "o-", ms=3); ax[0].set_title("Scree plot")
ax[0].set_xlabel("Component"); ax[0].set_ylabel("Eigenvalue")
ax[1].plot(range(1, len(cum) + 1), cum); ax[1].axhline(0.95, color="r", ls="--", label="95%")
ax[1].set_xlabel("Number of components"); ax[1].set_ylabel("Cumulative explained variance")
ax[1].set_title("Cumulative explained variance"); ax[1].legend(); ax[1].grid(alpha=0.3)
plt.tight_layout(); plt.show()

# ---------------- Task 4: components for 95% variance ----------------
k95 = int(np.argmax(cum >= 0.95) + 1)
print(f"\nComponents needed for 95% variance: {k95} (out of {d}) -> {d / k95:.1f}x compression")

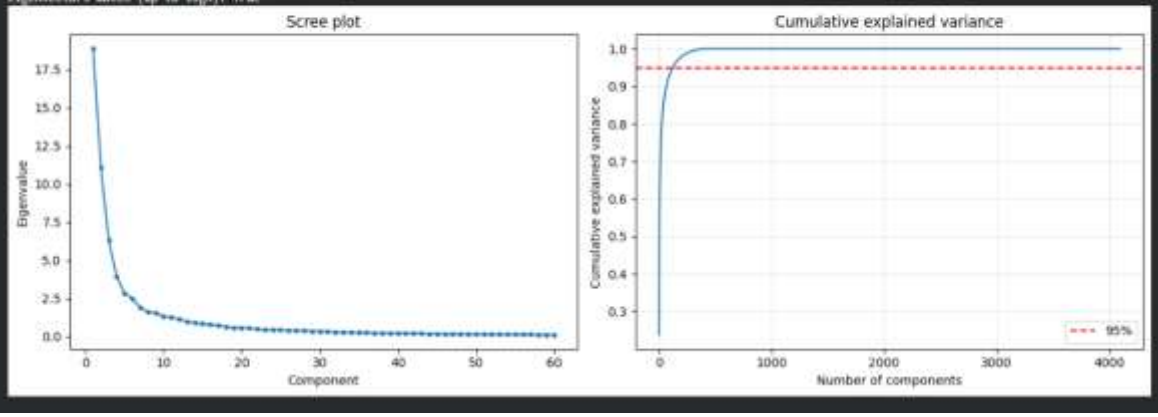

# ---------------- Task 5: eigenfaces and reconstructions ----------------
fig, axes = plt.subplots(2, 5, figsize=(12, 5.5))
for i, a in enumerate(axes.ravel()):
    a.imshow(eigvecs[:, i].reshape(IMG), cmap="gray"); a.set_title(f"Eigenface {i+1}", fontsize=9);
a.axis("off")
plt.suptitle(f"Top 10 eigenfaces ({NAME})"); plt.tight_layout(); plt.show()

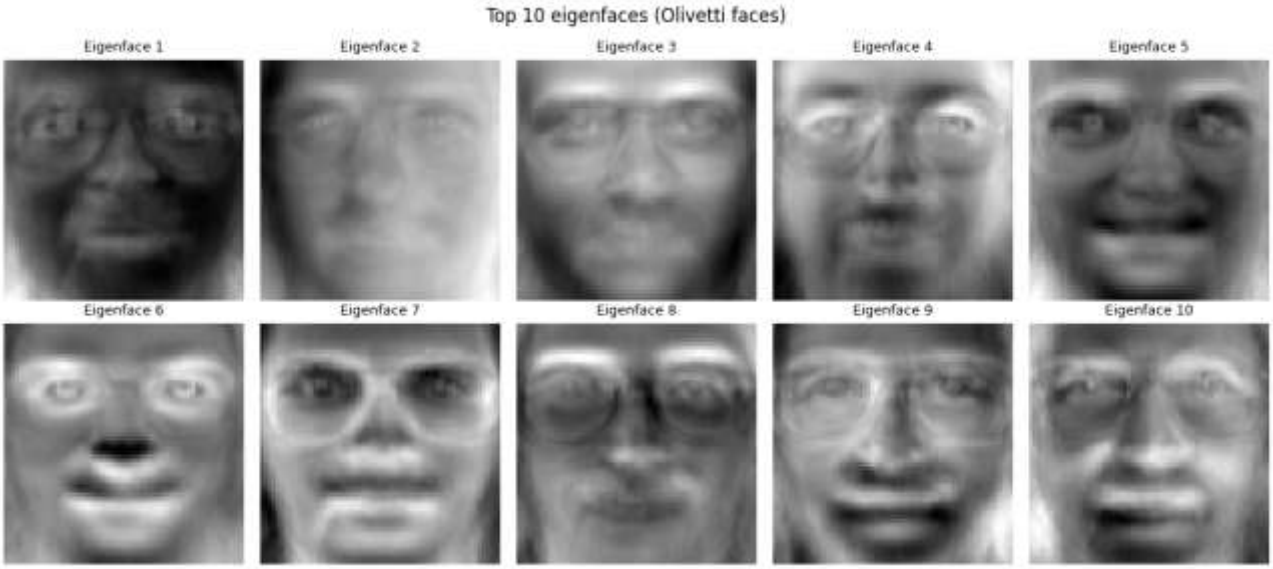
show = 5
fig, axes = plt.subplots(1 + len(KS), show, figsize=(2.2 * show, 2.3 * (1 + len(KS))))
for j in range(show):
    axes[0, j].imshow(X[j].reshape(IMG), cmap="gray"); axes[0, j].axis("off")
    if j == 0: axes[0, j].set_title("Original", fontsize=9, loc="left")
    for r, k in enumerate(KS, start=1):
        rec = reconstruct(Xc[j:j + 1], k)[0]
        axes[r, j].imshow(rec.reshape(IMG), cmap="gray"); axes[r, j].axis("off")
        if j == 0: axes[r, j].set_title(f"{k} components", fontsize=9, loc="left")
plt.suptitle("Reconstruction with increasing numbers of components"); plt.tight_layout(); plt.show()
for k in KS:
    mse = np.mean((reconstruct(Xc, k) - X) ** 2)
    print(f"k = {k:3d}: reconstruction MSE = {mse:.5f}, variance retained = {cum[k-1]*100:.1f}%")
```

Output

```
Downloading Olivetti faces from https://ndownloader.figshare.com/files/5976027 to /root/scikit_learn_data
Olivetti faces: 400 images, 4096 pixels each
From-scratch PCA took 51.0s

Variance ratio (mine)    : [0.23813 0.13994 0.07969 0.04998 0.0361 ]
Variance ratio (sklearn) : [0.23813 0.13994 0.07969 0.04998 0.0361 ]
Explained variance match : True
Eigenvectors match (up to sign): True
```








```
k =  10: reconstruction MSE = 0.00662, variance retained = 65.6%
k =  50: reconstruction MSE = 0.00243, variance retained = 87.4%
k = 100: reconstruction MSE = 0.00125, variance retained = 93.5%
```